

Exit Parameter Calculation — Method & Formula Report

Project: **fin-agent** | Module: Trader's Edge Exit P/E & IRR | Date: 23 Jul 2026

1. Executive Summary

The project produces an **Exit P/E** multiple for each stock and then derives an **Exit Price** and **projected IRR**. Two independent engines can predict the Exit P/E:

- **A. Trader's Edge Model (deterministic).** A closed-form, mean-reversion formula with a quality overlay. Fast, auditable, ~100–500 ms per batch. Source: `backend/exit_pe_model.py`.
- **B. GPT-based Model (o3, formula-constrained).** Same style of formula but executed by the LLM with a strict system prompt and JSON schema. Used inside the Top-IRR endpoint. Source: `backend/main_gpt_all.py :: run_gpt_predict_exit_pe_batched`.

Both engines feed the same downstream step: **Exit Price = EPS × Exit P/E** and **IRR = (Exit Price / Current Price)^{1/H} – 1**. When Exit P/E cannot be produced, a fallback of **18.0x** is used (market average).

2. Input Parameter Glossary (used by any engine)

Symbol / Field	Meaning	Units	Source Field
current_pe	Current forward P/E ratio	x (multiple)	pe_ratio
pe_avg_10y	10-year average P/E (outliers >100 excluded)	x	pe_avg_10y
pe_std	Std deviation of historical P/E	x	pe_std_10y
years_of_data	Valid P/E years used in the 10Y average	years	pe_avg_10y_years_used
roe	Return on Equity	%	roe
ebit_margin	EBIT / Operating Margin (moat indicator)	%	ebit_margin
eps_growth	3-year EPS CAGR	%	eps_cagr_3y
debt_ebitda	Debt / EBITDA (leverage) — GPT engine only	x	debt_ebitda
eps_forward	Forward EPS used to project Exit Price	currency / share	eps_forward
current_price	Latest traded price (DB or Stooq)	currency / share	current_price
horizon_years (H)	Forecast horizon for IRR	years	payload.horizon_years
pe_max / pe_min	Business clamps on Exit P/E	x	EXIT_PE_MAX env (80) / 3

3. Engine A — Trader's Edge Deterministic Model

Approach: measure how far the current valuation sits from its own 10-year history (z-score), then **revert a fraction of that gap** back toward the historical average. The reversion strength depends on how stretched the multiple is, and is dampened for high-quality names and amplified for low-quality names.

3.1 Step 1 — Z-Score (distance from history)

$$z = (\text{current_pe} - \text{pe_avg_10y}) / \text{pe_std}$$

If *pe_std* is missing or ≤ 1 , the model estimates it as **25% of *pe_avg_10y***. The sign of *z* decides direction: positive → overvalued (compress toward mean); negative → undervalued (expand toward mean).

3.2 Step 2 — Valuation Regime Label

Condition	Regime
$z > +1.5$	OVERVALUED
$z < -1.5$	UNDERVALUED
otherwise	FAIR_VALUE

3.3 Step 3 — Quality Multiplier (dampens or amplifies reversion)

A per-name quality score is built from three fundamentals. Each contributes a point score; the total is converted to a multiplier that scales the reversion rate.

Factor	Threshold	Points added to quality_score
ROE	> 25 %	+0.25
ROE	> 15 % and $\leq$ 25 %	+0.15
ROE	< 8 %	−0.15
EBIT Margin	> 20 %	+0.20
EBIT Margin	> 12 % and $\leq$ 20 %	+0.10
EBIT Margin	< 5 %	−0.15
EPS Growth (3Y)	> 20 %	+0.20
EPS Growth (3Y)	> 10 % and $\leq$ 20 %	+0.10
EPS Growth (3Y)	< 0 %	−0.15

$$\text{quality_mult} = \text{clamp}(1 - \text{quality_score} \times 0.8, 0.5, 1.4)$$

Interpretation: high-quality names get a multiplier below 1 (revert less — premium holds); low-quality names get a multiplier above 1 (revert more).

Descriptive labels:

- $\text{quality_score} > 0.25 \rightarrow$ **High Quality**
- $\text{quality_score} > 0.05 \rightarrow$ **Above Average**
- $\text{quality_score} > -0.10 \rightarrow$ **Average**
- else $\rightarrow$ **Below Average**

3.4 Step 4 — Reversion Rate Buckets (by |z|)

The base reversion rate is chosen from a lookup table on the magnitude of the z-score, then multiplied by the quality multiplier. Calibrated on 3,161 company-year observations (2013–2024): Q5 (highest-PE quintile) compresses ~33% on average, Q1 expands ~22%.

z bucket	Base rate	Explanation
< 0.5	5 %	Near historical average
0.5 – 1.0	10 %	Slight deviation
1.0 – 1.5	15 %	Moderate deviation
1.5 – 2.0	20 %	Significant deviation
2.0 – 2.5	25 %	Stretched
2.5 – 3.0	35 %	Very stretched
≥ 3.0	45 %	Extreme outlier

`reversion_rate = base_rate × quality_mult`

3.5 Step 5 — Exit P/E Formula

If $z > 0$ (overvalued):

`exit_pe = current_pe - reversion_rate × (current_pe - pe_avg_10y)`

If $z \leq 0$ (undervalued):

`exit_pe = current_pe + reversion_rate × (pe_avg_10y - current_pe)`

Then clamped: `exit_pe = clamp(exit_pe, pe_min=3, pe_max=80)`

In plain English: take a fraction of the gap between the current multiple and its long-run average, and close it. The fraction is set by z-bucket and quality.

3.6 Step 6 — Confidence Band around Exit P/E

`band_width_% = 0.15 + min(|z|×0.03, 0.15) + (1 - data_quality)×0.10 + (0.05 if exit_pe > 40 else 0)`
`half_width = exit_pe × band_width_%`
`exit_pe_low = max(3, exit_pe - half_width)`
`exit_pe_high = min(100, exit_pe + half_width)`
`confidence = clamp(0.85 - z_uncertainty - data_uncertainty, 0.30, 0.95)`

where **data_quality = min(years_of_data / 10, 1.0)**. Wider bands are produced for (a) larger z, (b) fewer historical years, and (c) very high multiples (> 40x).

Conviction label:

- confidence ≥ 0.75 and $|z| < 1.5 \rightarrow$ **High**
- confidence ≥ 0.55 $\rightarrow$ **Medium**
- else $\rightarrow$ **Low**

3.7 Worked Example

Inputs: current_pe = 30x, pe_avg_10y = 20x, pe_std = 4, ROE = 22%, EBIT Margin = 18%, EPS Growth = 12%, years_of_data = 10.

$z = (30 - 20) / 4 = +2.5 \rightarrow$ Regime = OVERVALUED, Bucket = 2.5–3.0 $\rightarrow$ base_rate = 35 %.
Quality: ROE +0.15, Margin +0.10, Growth +0.10 $\rightarrow$ quality_score = 0.35 $\rightarrow$ quality_mult = `clamp(1 - 0.35×0.8, 0.5, 1.4) = 0.72`.
`reversion_rate = 0.35 × 0.72 = 25.2 %`.
`exit_pe = 30 - 0.252 × (30 - 20) = 30 - 2.52 = 27.48x`.
Band: $0.15 + \min(2.5 \times 0.03, 0.15) + 0 + 0 = 0.225 \rightarrow$ half = 6.18 $\rightarrow$ range **21.3x - 33.7x**.
Confidence ≈ 0.78 (but $|z| \geq 1.5 \rightarrow$ Conviction **Medium**).

4. Engine B — GPT (o3) Formula-Constrained Model

Used inside `/api/top-irr` when the deterministic engine is not selected. The LLM receives structured fields and is *forced* to follow the formula below and return audit-grade JSON. Model is set by env **GPT_MODEL** (default: **o3**).

4.1 Base Multiple

```
If pe_avg_10y is available (years_used > 0):
    BaseMultiple = pe_avg_10y
Else:
    BaseMultiple = 18.0 (conservative market-average proxy)
```

4.2 Component Scores (clamped, missing → 0)

```
GrowthScore = clamp(eps_cagr_3y, -10, 30) / 30
RoeScore = clamp(roe, 0, 30) / 30
MarginScore = clamp(ebit_margin, 0, 25) / 25
ProfitScore = 0.6 × RoeScore + 0.4 × MarginScore
LevScore = 1 - clamp(debt_ebitda, 0, 6) / 6
```

4.3 Weighted Multiplier

```
Multiplier = 1 + 0.25 × GrowthScore + 0.15 × (ProfitScore - 0.5) + 0.10 × (LevScore - 0.5)
```

Component	Weight	Centered at	Range of contribution
GrowthScore	0.25	0 (raw)	-0.083 ... +0.25
ProfitScore	0.15	0.5	-0.075 ... +0.075
LevScore	0.10	0.5	-0.05 ... +0.05

4.4 Final Exit P/E & Confidence

```
exit_pe = clamp(BaseMultiple × Multiplier, 3, EXIT_PE_MAX)
EXIT_PE_MAX defaults to 80 (env EXIT_PE_MAX).

confidence = 0.85 - 0.10 × (# of NA among pe_avg_10y, roe, ebit_margin, debt_ebitda,
eps_cagr_3y)
confidence = clamp(confidence, 0.25, 0.90)
```

4.5 Post-Processing Guard

After the LLM returns a value, the server enforces the same clamp with `_clamp_exit_pe()`: **max(3.0, min(exit_pe, EXIT_PE_MAX))**. This prevents any model drift from breaching business bounds.

5. Downstream — Exit Price & IRR

Once Exit P/E is set (from either engine), the same two lines produce the trade-level output:

```
Exit Price = EPS (forward) × Exit P/E
IRR = (Exit Price / Current Price) ^ (1 / horizon_years) - 1
```

Current Price defaults to the DB price and is optionally refreshed via Stooq for the top-N candidates (env USE_STOOQ_PRICE=1). horizon_years is user-supplied and clamped to **1 ... 15**.

6. Fallback Rules

Condition	Behavior
current_pe missing / ≤ 0	current_pe := 18.0
pe_avg_10y missing / ≤ 0	pe_avg_10y := 18.0
pe_std missing / ≤ 1	pe_std := $0.25 \times \text{pe_avg_10y}$
Trader's Edge cannot run (no current_pe or pe_avg_10y)	current_pe := 18.0x, band 15–22x, conviction Low
Any fundamental (ROE / Margin / Growth / Debt) missing	missing as neutral (0) in GPT engine; skipped in quality overlay in Trader's Edge
EPS or current_price missing	Ticker skipped in IRR ranking

7. Engine Comparison at a Glance

Aspect	Trader's Edge (deterministic)	GPT (o3)
Anchor	10Y avg P/E (mean-reversion)	10Y avg P/E (level)
Adjustment lever	Reversion rate × quality mult	Weighted multiplier on Base
Fundamentals used	ROE, EBIT Margin, EPS Growth	ROE, EBIT Margin, EPS Growth, Debt/EBITDA
Direction logic	Toward historical mean	Bottom-up score-driven
Confidence output	Band + High/Med/Low	0..1 scalar
Clamp	3 – 80x (env)	3 – 80x (env)
Latency	~100–500 ms full universe	Minutes (LLM batches of 50)
Determinism	Fully deterministic	Model-dependent, formula-constrained

8. Analyst Notes

- Both engines **anchor to the stock's own history**, not a sector median. Sector median P/E is explicitly excluded from the GPT prompt to avoid look-through bias.
- The Trader's Edge model gives an **explicit range** (low/high). Use the low bound for downside IRR and the high bound for upside IRR when building scenario decks.
- Exit P/E is capped by **EXIT_PE_MAX** (default 80). Adjust in the deployment env to loosen or tighten how aggressive multiples can go for high-growth names.
- The IRR formula compounds annually. For sub-year horizons, use horizon_years = 1 (the system already clamps to ≥ 1).

Sources: backend/exit_pe_model.py, backend/main_traders_edge.py, backend/main_gpt_all.py.